Docker for Microservices

Complete Guide with CI/CD & Canary Deployment

Docker Kubernetes CI/CD Canary

Version 1.0

July 2026

Table of Contents

1. Docker for DevOps - Introduction
2. Docker Container Definition
3. Docker Architecture
4. Docker Interactions
5. Docker Installation
6. Basic Docker Commands
7. Dockerfile
8. Creating Dockerfiles
9. Push Docker Image to Registry
10. CI/CD Pipeline with Canary Deployment

1. Docker for DevOps - Introduction

Definition

Docker for DevOps is the application of containerization technology to streamline the DevOps lifecycle. It enables developers to package applications and dependencies into standardized units, ensuring consistency across development, testing, and production environments.

Key Concepts

- **Containerization:** Packaging code with its dependencies
- **DevOps Integration:** CI/CD pipelines, automated testing
- **Infrastructure as Code:** Declarative infrastructure management
- **Portability:** Run anywhere - local, cloud, on-premise

Hands-On Lab: Docker Introduction

```
# 1. Verify Docker is installed
docker --version

# 2. Run your first container
docker run hello-world

# 3. Run an interactive container
docker run -it ubuntu:latest bash

# 4. Run a web server
docker run -d -p 8080:80 nginx

# 5. Verify it's running
curl http://localhost:8080
```

Lab Complete: You've successfully installed Docker and run your first containers!

2. Docker Container Definition

Definition

Docker Container is a lightweight, standalone, executable package that includes everything needed to run software: code, runtime, system tools, libraries, and settings. It's a "shipping container system for code" that enables applications to run consistently across any environment.

Key Characteristics

- **Lightweight:** Shares host OS kernel
- **Portable:** Runs on any Docker-enabled system
- **Self-contained:** Includes all dependencies
- **Isolated:** Runs in separate namespaces

Hands-On Lab: Container Operations

```
# 1. List all containers
docker ps -a

# 2. Create and run a container
docker run -d --name web-server nginx

# 3. Stop the container
docker stop web-server

# 4. Start the container
docker start web-server

# 5. Restart the container
docker restart web-server

# 6. Execute commands inside container
docker exec -it web-server bash

# 7. View logs
docker logs web-server
```

Tip: The `-d` flag runs the container in detached mode (background), and `-it` runs it interactively with a terminal.

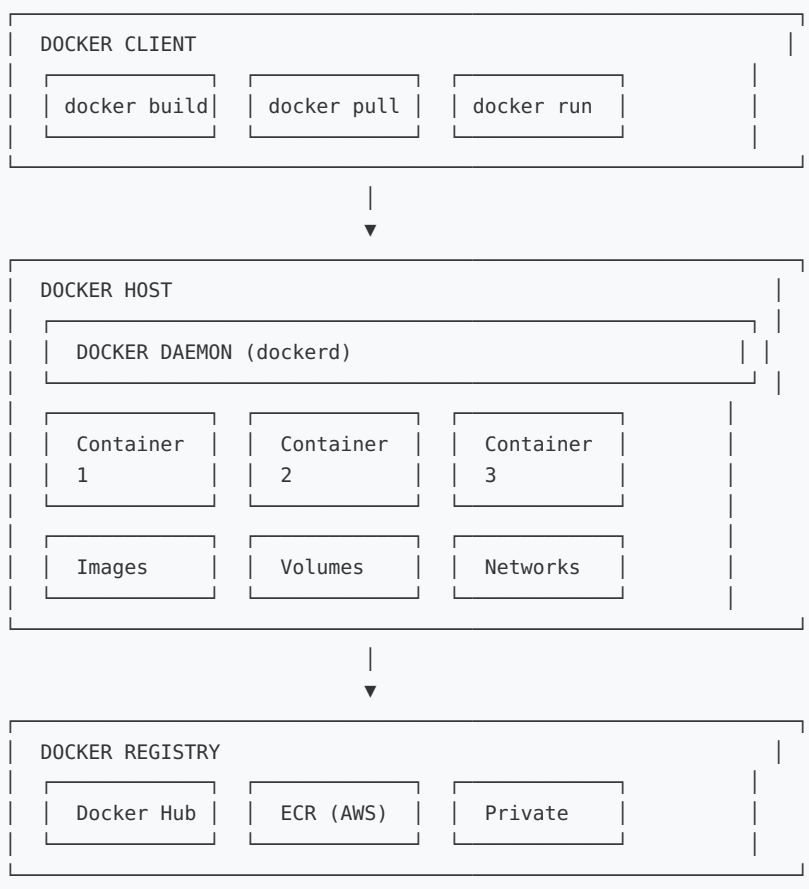
3. Docker Architecture

Definition

Docker Architecture follows a client-server model where:

- **Client:** Command-line interface (CLI) that sends commands
- **Server:** Docker daemon (dockerd) that manages containers
- **Registry:** Stores Docker images
- **Host:** System where containers run

Architecture Diagram



Hands-On Lab: Understanding Docker Architecture

```
# 1. Check Docker daemon status
sudo systemctl status docker
docker info

# 2. Check Docker client configuration
docker version

# 3. View Docker daemon logs
sudo journalctl -u docker.service -f

# 4. Inspect container networking
docker network ls
docker network inspect bridge
```

```
# 5. Check storage driver
docker info | grep "Storage Driver"
```

4. Docker Interactions

Definition

Docker Interactions define how Docker components communicate:

- **Client ↔ Daemon:** REST API communication
- **Daemon ↔ Registry:** Push/pull images
- **Daemon ↔ Containers:** Manage container lifecycle
- **Containers ↔ Volumes:** Persistent data storage
- **Containers ↔ Networks:** Communication between containers

Hands-On Lab: Docker Interactions

```
# 1. BUILD - Create an image from Dockerfile
docker build -t myapp:latest .

# 2. RUN - Create container from image
docker run myapp:latest

# 3. COMMIT - Create image from container
docker commit container_id myapp-with-curl

# 4. PUSH/PULL - Registry operations
docker tag myapp:latest username/myapp:latest
docker push username/myapp:latest

# 5. START/STOP/RESTART - Container lifecycle
docker run -d --name demo nginx
docker stop demo
docker start demo
docker restart demo

# 6. VOLUMES - Persistent data
docker volume create mydata
docker run -v mydata:/data alpine:latest touch /data/test.txt
```

5. Docker Installation

Definition

Docker Installation is the process of setting up Docker Engine on a host system. It involves adding the official Docker repository, installing the Docker packages, and configuring the Docker daemon.

Installation Steps (Ubuntu)

```
#!/bin/bash
# Complete Docker Installation Script

# 1. Update existing packages
sudo apt-get update

# 2. Install prerequisites
sudo apt-get install -y \
    apt-transport-https \
    ca-certificates \
    curl \
    gnupg \
    lsb-release

# 3. Add Docker's official GPG key
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

# 4. Add Docker repository
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] \
    https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
    sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 5. Update package index
sudo apt-get update

# 6. Install Docker Engine
sudo apt-get install -y docker-ce docker-ce-cli containerd.io

# 7. Start Docker daemon
sudo systemctl start docker
sudo systemctl enable docker

# 8. Verify installation
docker --version
sudo systemctl status docker --no-pager

# 9. Add user to docker group
sudo usermod -aG docker $USER

# 10. Test installation
docker run hello-world
```

⚠ **Note:** You need to log out and log back in for the group changes to take effect.

6. Basic Docker Commands

Definition

Basic Docker Commands are essential commands for interacting with Docker's core functionality, including image management, container lifecycle, and system administration.

Image Management Commands

Command	Description	Example
`docker images`	List images	`docker images -a`
`docker pull`	Pull image from registry	`docker pull nginx:latest`
`docker build`	Build image from Dockerfile	`docker build -t myapp .`
`docker rmi`	Remove image	`docker rmi myapp`
`docker tag`	Tag image	`docker tag myapp:latest user/myapp:1.0`

Container Management Commands

Command	Description	Example
`docker run`	Run container	`docker run -d -p 8080:80 nginx`
`docker ps`	List running containers	`docker ps -a`
`docker stop`	Stop container	`docker stop container_id`
`docker start`	Start container	`docker start container_id`
`docker restart`	Restart container	`docker restart container_id`
`docker rm`	Remove container	`docker rm -f container_id`
`docker exec`	Execute inside container	`docker exec -it container_id bash`
`docker logs`	View logs	`docker logs -f container_id`
`docker inspect`	Inspect container	`docker inspect container_id`

Hands-On Lab: Complete Command Practice

```
# Pull an image
docker pull nginx:alpine

# Run a container with port mapping
docker run -d -p 8080:80 --name webapp nginx:alpine

# Check if it's running
docker ps

# View logs
docker logs webapp

# Execute inside container
docker exec -it webapp sh

# Stop and remove
docker stop webapp
docker rm webapp
```

7. Dockerfile

Definition

Dockerfile is a text file containing instructions to build a Docker image. It defines the environment, dependencies, and configuration for the application.

Dockerfile Instructions Reference

Instruction	Description	Example
`FROM`	Base image	`FROM openjdk:11-jre-slim`
`LABEL`	Metadata	`LABEL maintainer="team@example.com"`
`ENV`	Environment variables	`ENV APP_HOME=/app`
`WORKDIR`	Working directory	`WORKDIR /app`
`RUN`	Execute commands during build	`RUN apt-get update`
`COPY`	Copy files from host	`COPY . /app`
`EXPOSE`	Declare ports	`EXPOSE 8080`
`USER`	Switch user	`USER appuser`
`CMD`	Default command	`CMD ["java", "-jar", "app.jar"]`
`ENTRYPOINT`	Fixed command	`ENTRYPOINT ["java", "-jar"]`
`HEALTHCHECK`	Health check	`HEALTHCHECK CMD curl -f http://localhost || exit 1`

Hands-On Lab: Creating Dockerfiles

```
# Create a simple Dockerfile
cat > Dockerfile << 'EOF'
FROM alpine:latest
LABEL maintainer="dev@example.com"
ENV APP_NAME=HelloWorld
WORKDIR /app
COPY hello.sh .
RUN chmod +x hello.sh
CMD ["./hello.sh"]
EOF

# Create the script
cat > hello.sh << 'EOF'
#!/bin/sh
echo "Hello from $APP_NAME!"
EOF

# Build and run
docker build -t hello-app:latest .
docker run hello-app:latest
```

8. Creating Dockerfiles

Definition

Creating Dockerfiles involves writing instructions to build custom images for different application types, ensuring proper layering and optimization.

Example 1: Simple Webserver

```
# webserver/Dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

# Build and run
docker build -t webserver:latest .
docker run -d -p 8080:80 webserver:latest
```

Example 2: Java Maven Application

```
# Multi-stage Dockerfile
FROM maven:3.8.4-openjdk-11-slim AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

FROM openjdk:11-jre-slim
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

Example 3: Spring Boot Microservice

```
FROM maven:3.8.4-openjdk-11-slim AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

FROM openjdk:11-jre-slim
WORKDIR /app
RUN apt-get update && apt-get install -y curl
COPY --from=builder /app/target/*.jar app.jar
RUN groupadd -r app && useradd -r -g app appuser
USER appuser
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s \
  CMD curl -f http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Best Practice: Use multi-stage builds to keep images small and secure. Always create a non-root user for production containers.

9. Push Docker Image to Registry

Definition

Pushing Docker Images to Registry involves uploading built images to a container registry (Docker Hub, AWS ECR, GCR, etc.) for distribution and deployment.

Docker Hub Setup

```
# 1. Login to Docker Hub
docker login

# 2. Tag your image
docker tag myapp:latest username/myapp:latest
docker tag myapp:latest username/myapp:1.0.0

# 3. Push to Docker Hub
docker push username/myapp:latest
docker push username/myapp:1.0.0

# 4. Pull from Docker Hub
docker pull username/myapp:latest
```

Private Registry (AWS ECR)

```
# 1. Get login password
aws ecr get-login-password --region us-east-1 | \
    docker login --username AWS --password-stdin \
    account-id.dkr.ecr.us-east-1.amazonaws.com

# 2. Tag image
docker tag myapp:latest account-id.dkr.ecr.us-east-1.amazonaws.com/myapp:latest

# 3. Push to ECR
docker push account-id.dkr.ecr.us-east-1.amazonaws.com/myapp:latest
```

Hands-On Lab: Complete Registry Workflow

```
#!/bin/bash
# Complete Registry Workflow

# Configuration
REGISTRY="docker.io"
USERNAME="yourusername"
APP_NAME="myapp"
VERSION="1.0.0"
IMAGE_NAME="${REGISTRY}/${USERNAME}/${APP_NAME}:${VERSION}"

# 1. Build the image
docker build -t ${APP_NAME}:${VERSION} .

# 2. Login to registry
docker login ${REGISTRY}

# 3. Tag the image
docker tag ${APP_NAME}:${VERSION} ${IMAGE_NAME}

# 4. Push to registry
docker push ${IMAGE_NAME}
```

```
# 5. Verify
docker pull ${IMAGE_NAME}
docker run --rm ${IMAGE_NAME} echo "Success!"

echo " Image pushed successfully!"
```

10. CI/CD Pipeline with Canary Deployment

Overview

A complete CI/CD pipeline with canary deployment enables safe, gradual rollouts of new versions with automatic rollback capabilities.

Canary Deployment Flow

```
Phase 1: 10% Traffic → Monitor (2 min)
Phase 2: 50% Traffic → Monitor (2 min)
Phase 3: 100% Traffic → Full Deployment
Phase 4: Rollback if errors detected
```

Kubernetes Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-stable
  labels:
    app: myapp
    track: stable
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
      track: stable
  template:
    metadata:
      labels:
        app: myapp
        track: stable
    spec:
      containers:
      - name: myapp
        image: ${REGISTRY}/${IMAGE_NAME}:${VERSION}
        ports:
        - containerPort: 8080
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        livenessProbe:
          httpGet:
            path: /actuator/health/liveness
            port: 8080
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /actuator/health/readiness
            port: 8080
          initialDelaySeconds: 10
          periodSeconds: 5
```

GitHub Actions CI/CD Pipeline

```
name: CI/CD Pipeline with Canary Deployment

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

env:
  REGISTRY: docker.io
  IMAGE_NAME: ${{ github.repository }}
  KUBE_NAMESPACE: production

jobs:
  # CI Job
  ci:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout Code
      uses: actions/checkout@v3

    - name: Build Application
      run: mvn clean package -DskipTests

    - name: Build Docker Image
      run: |
        docker build -t ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }} .

    - name: Push Docker Image
      run: |
        docker push ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}

  # CD Job with Canary Deployment
  cd:
    needs: ci
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
    - name: Deploy Canary (10%)
      run: kubectl set image deployment/app-canary \
        myapp=${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}

    - name: Monitor Canary
      run: |
        sleep 120
        ERROR_COUNT=$(kubectl logs -l app=myapp,track=canary | grep -c ERROR)
        if [ "$ERROR_COUNT" -gt 10 ]; then
          echo " High error rate! Rolling back..."
          exit 1
        fi

    - name: Full Rollout (100%)
      run: kubectl apply -f virtual-service-100.yaml
```

Complete Deployment Script

```
#!/bin/bash
# deploy-with-canary.sh

set -e
```

```
APP_NAME="myapp"
REGISTRY="docker.io"
USERNAME="yourusername"
VERSION="1.0.${BUILD_NUMBER}"
NAMESPACE="production"

# Deploy Stable
kubectl set image deployment/app-stable \
    myapp=${REGISTRY}/${USERNAME}/${APP_NAME}:${VERSION} \
    -n ${NAMESPACE}
kubectl rollout status deployment/app-stable -n ${NAMESPACE}

# Deploy Canary (10%)
kubectl set image deployment/app-canary \
    myapp=${REGISTRY}/${USERNAME}/${APP_NAME}:${VERSION} \
    -n ${NAMESPACE}
kubectl rollout status deployment/app-canary -n ${NAMESPACE}

# Route 10% Traffic
kubectl apply -f virtual-service-10.yaml

# Monitor Canary
sleep 120
ERROR_COUNT=$(kubectl logs -l app=myapp,track=canary -n ${NAMESPACE} | grep -c ERROR || echo "0")
if [ "$ERROR_COUNT" -gt 10 ]; then
    echo " Canary errors detected! Rolling back..."
    kubectl rollout undo deployment/app-canary -n ${NAMESPACE}
    exit 1
fi

# Route 50% Traffic
kubectl apply -f virtual-service-50.yaml

# Monitor Canary
sleep 120
ERROR_COUNT=$(kubectl logs -l app=myapp,track=canary -n ${NAMESPACE} | grep -c ERROR || echo "0")
if [ "$ERROR_COUNT" -gt 10 ]; then
    echo " Canary errors detected! Rolling back..."
    kubectl rollout undo deployment/app-canary -n ${NAMESPACE}
    exit 1
fi

# Full Rollout (100%)
kubectl apply -f virtual-service-100.yaml
kubectl scale deployment/app-stable --replicas=0 -n ${NAMESPACE}

echo " Deployment Complete!"
```

Deployment Complete! The canary deployment strategy ensures zero-downtime rollouts with automatic rollback capabilities.

Summary

This comprehensive guide covers:

- **Docker Basics:** Overview, architecture, installation
- **Docker Commands:** Complete command reference with examples
- **Dockerfile:** All instructions and hands-on examples
- **Container Management:** Build, run, and manage containers
- **Registry Operations:** Push/pull to Docker Hub and private registries

- **CI/CD Pipeline:** Complete automation with GitHub Actions
- **Canary Deployment:** Gradual rollout with Kubernetes

© 2026 - All Rights Reserved